

DOCUMENTO TÉCNICO INTERNO

CR Costa Rica · Odoo Community 19.0 · API_Hacienda (CRLibre)

Despliegue de la API CRLibre en DigitalOcean

Arquitectura centralizada para hospedar el servicio de Facturación Electrónica y servir a múltiples clientes de Odoo personalizado.

Versión	Fecha	Alcance
1.0	2026-07-01	Arquitectura, despliegue, costos y riesgos

Tabla de Contenidos

1. Resumen Ejecutivo

2. Contexto: por qué centralizar la API

3. Justificación del Stack Tecnológico

4. Arquitectura General

5. Multi-Tenencia: Cómo Sirve a Varios Clientes

6. Stack de DigitalOcean Recomendado

7. Guía de Despliegue

8. Seguridad y Certificados .p12

9. Precios Estimados

10. Riesgos y Aspectos a Considerar

11. Ruta de Escalamiento Futuro

12. Conclusión y Próximos Pasos

SECCIÓN 1

Resumen Ejecutivo

Este documento define la arquitectura recomendada para hospedar la **API_Hacienda de CRLibre** —el componente que genera y firma comprobantes de Factura Electrónica para Costa Rica— como un **servicio centralizado propio**, en vez de instalarla localmente en la máquina de cada cliente. Cada instalación de Odoo (una por cliente, local en su negocio) se conectará a esta API centralizada por internet.

Recomendación

Nube:

DigitalOcean

· Cómputo:

un Droplet + Docker Compose

· Proxy/TLS:

Caddy

· Base de datos:

MariaDB en contenedor

· Costo inicial estimado:

USD 20–35/mes

.

Esta elección prioriza simplicidad operativa y menor curva de aprendizaje para un equipo pequeño, reutilizando la misma configuración de Docker Compose que ya fue construida y validada en el entorno local durante la prueba de concepto (PoC) de integración Odoo ↔ CRLibre.

Punto crítico a resolver antes de producción

En la PoC local, los endpoints de la API (`clave`, `gen_xml_fe`) se usaron en modo `users_openAccess` (sin autenticación) porque todo corría dentro de la misma máquina. Al exponer la API a internet para múltiples clientes,

esto debe cambiar

: se requiere autenticación por cliente antes de aceptar tráfico público. Ver Sección 5.

SECCIÓN 2

Contexto: por qué centralizar la API

Se evaluaron dos modelos de despliegue para la API de CRLibre en un escenario donde el producto (Odoo personalizado) se vende a múltiples clientes:

Opción A — API local en cada cliente

- ✗ Cada cliente requiere un stack extra (PHP + Apache + MariaDB + Docker) que hay que soportar remotamente.
- ✗ Actualizar la lógica de negocio implica actualizar N máquinas.
- ✗ Mayor probabilidad de fallas de infraestructura por instalación (ya se observaron 4 problemas distintos solo para levantar el stack una vez).

Opción B — API centralizada (elegida)

- ✓ Un solo lugar para actualizar, parchar y monitorear.
- ✓ El cliente solo necesita Odoo local + internet.
- ✓ Menor superficie de soporte técnico por cliente nuevo.

Un dato clave que valida esta decisión: **el envío final del comprobante a Hacienda siempre requiere internet**, sin importar dónde corra la API, porque el endpoint de recepción (`api.comprobanteselectronicos.go.cr`) es de Hacienda, en la nube. La ventaja de "todo local y offline" de la Opción A no aplica al paso más crítico del proceso de todos modos, lo que inclina la balanza hacia centralizar.

Justificación del Stack Tecnológico

3.1 Por qué DigitalOcean (y no AWS, GCP o Azure)

Para el volumen esperado (una API PHP liviana, con llamadas JSON simples, sirviendo a un número moderado de clientes), **no se justifica** la complejidad operativa de un proveedor hyperscale. DigitalOcean ofrece precio fijo y predecible, la mejor documentación para Docker Compose entre las opciones simples, y una curva de aprendizaje baja para un equipo pequeño.

Proveedor	Veredicto	Motivo
DigitalOcean	Elegido	Simplicidad, precio fijo, Docker nativo, curva de aprendizaje baja.
AWS Lightsail	Alternativa válida	Similar en simplicidad; tiene sentido solo si ya se usa el ecosistema AWS.
Google Cloud / Firebase	Descartado	Firebase no soporta PHP ni MySQL (es NoSQL); GCP "puro" es viable pero más complejo sin necesidad.
Azure	Descartado	Sobre-ingeniería para un contenedor PHP + un contenedor MariaDB.
Kubernetes gestionado	Descartado	Complejidad de orquestación innecesaria a esta escala.

3.2 Por qué Docker Compose (y no Kubernetes)

Un solo servidor ejecutando 2-3 contenedores es más simple de operar, depurar y razonar para un equipo pequeño que un clúster orquestado. Además, **reutiliza exactamente** la configuración ya construida y probada durante la PoC local, lo que reduce el riesgo de migración: se conocen de antemano los problemas de infraestructura de este stack específico (ver Sección 8 de la nota técnica de la PoC) y ya están resueltos.

3.3 Por qué Caddy (y no Nginx + Certbot manual)

Caddy renueva automáticamente los certificados TLS de Let's Encrypt sin configuración adicional. Esto elimina una clase entera de incidentes ("el certificado expiró y la facturación de todos los clientes se cayó") que sí es un riesgo real con una configuración manual de Nginx + Certbot mal mantenida.

3.4 Por qué MariaDB auto-hospedada en contenedor (y no una base de datos gestionada, al inicio)

El código de CRLibre usa `mysql_i` de forma directa —es una dependencia dura hacia MySQL/MariaDB, no hay alternativa—. Se recomienda iniciar con MariaDB en contenedor (igual que en la PoC) por costo y simplicidad; **DigitalOcean Managed Database** queda como ruta de mejora cuando el volumen de clientes lo justifique (ver Sección 11).

Consideración legal — Licencia AGPL v3

El código de CRLibre está bajo licencia

AGPL v3

. Esta licencia incluye una cláusula de "uso en red" (Sección 13): si se ejecuta una *versión modificada* del software y los usuarios interactúan con ella por red, existe la obligación de ofrecerles el código fuente correspondiente a esa versión modificada. Durante la PoC se modificaron archivos de infraestructura del propio repositorio de CRLibre (`Dockerfile`, `docker-entrypoint.sh`) para poder levantarlo. Al convertir esto en un **servicio de red comercial**

, esta cláusula deja de ser hipotética.

Se recomienda consultar con un abogado especializado en licencias de software libre

antes del lanzamiento; una mitigación común es publicar el código modificado (los parches de infraestructura) en un repositorio accesible para quien lo solicite.

Arquitectura General

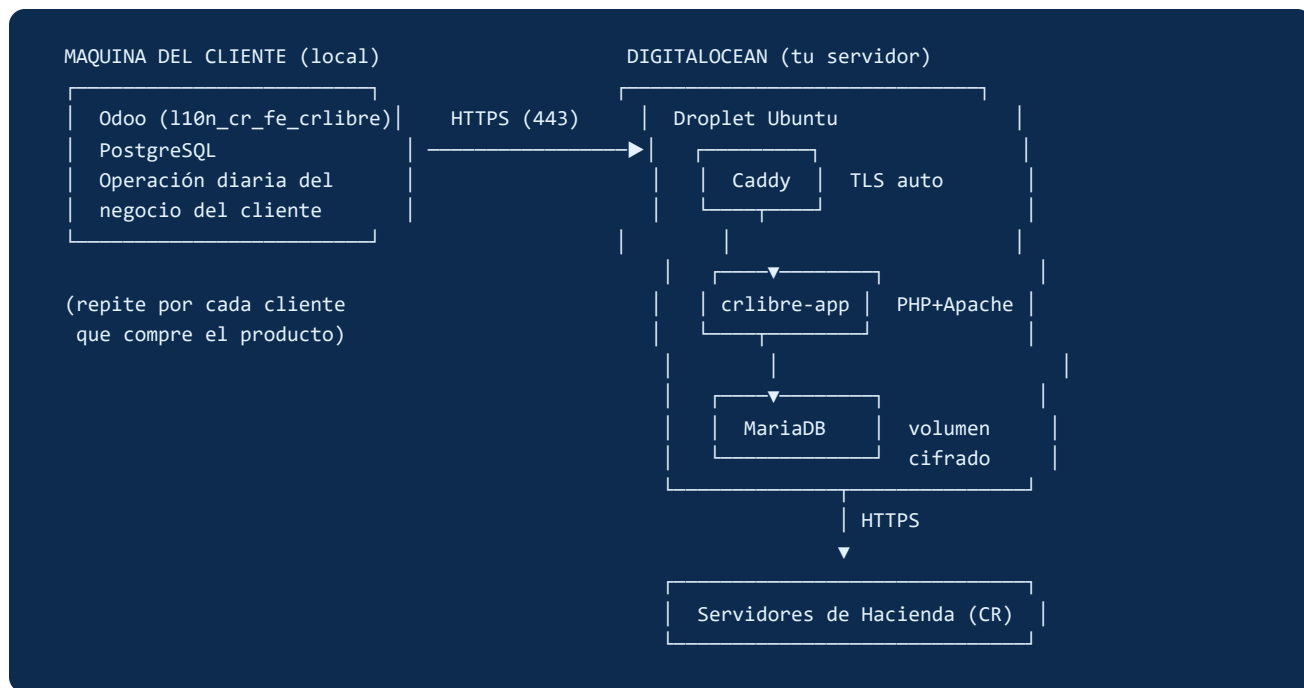


Figura 1 — Arquitectura centralizada: N clientes de Odoo local, un solo servidor CRLibre en DigitalOcean.

El único cambio necesario del lado de Odoo respecto a la PoC local es el valor del parámetro de configuración `110n_cr_fe.api_url`: pasa de `http://host.docker.internal:8080` a la URL pública del servidor, por ejemplo `https://fe-api.tuempresa.com`. El resto del addon no requiere cambios porque ya fue diseñado para leer la URL desde configuración.

SECCIÓN 5

Multi-Tenencia: Cómo Sirve a Varios Clientes

El propio código de CRLibre ya incluye un modelo de aislamiento de datos por cliente, observado directamente en `facturadorCRLibre.php`: usa un patrón de **"tabla por inquilino"**. Cada empresa registrada (`idUser` / `idMasterUser`) obtiene su propio juego de tablas, por ejemplo:

```
-- Ejemplo de tablas generadas para el cliente con idUser = 5
5_master_vouchers
5_master_config_companny
5_master_receiver
5_master_consecutive
5_master_inventory_sucursal_001
```

Esto significa que **una sola instalación de la API, en un solo servidor, puede servir a todos los clientes** sin necesidad de instalaciones separadas por cliente — cada empresa nueva simplemente agrega su propio conjunto de tablas dentro de la misma base de datos MariaDB.

Gap de seguridad a cerrar antes de producción

En la PoC, los endpoints usados (`w=clave&r=clave`, `w=genXML&r=gen_xml_fe`) están marcados en el código como `users_openAccess`: cualquiera que conozca la URL puede llamarlos, sin login. Esto fue intencional y correcto para una prueba de concepto aislada en `localhost`, pero es

inaceptable en un servicio multi-cliente expuesto a internet

: sin autenticación, un cliente podría (intencional o accidentalmente) generar comprobantes usando los datos de otro, o un tercero podría saturar el servicio.

Antes de aceptar tráfico público de múltiples clientes, es necesario:

- Migrar a los endpoints `users_loggedIn` / `companny_users_loggedIn` que ya existen en el código de CRLibre (usan `sessionKey` por usuario), o
- Agregar una capa de autenticación por API key a nivel del proxy (Caddy), asignando una key única por cliente de Odoo, o
- Ambas medidas combinadas, para defensa en profundidad.

SECCIÓN 6

Stack de DigitalOcean Recomendado

Capa	Servicio de DigitalOcean	Propósito
Cómputo	Droplet (2–4 GB RAM / 1–2 vCPU)	Corre el mismo <code>docker-compose.yml</code> ya validado en la PoC.
Proxy + TLS	Caddy (contenedor adicional)	HTTPS automático vía Let's Encrypt, sin gestión manual de certificados.
Base de datos	MariaDB en contenedor	Persistencia de claves, XML, consecutivos y datos por cliente.
Red / DNS	DigitalOcean DNS (gratis)	Apunta el dominio/subdominio a la IP del Droplet.
Firewall	Cloud Firewall (gratis)	Solo puerto 443 público; SSH restringido por IP; MariaDB nunca expuesta.
Backups del servidor	Droplet Backups (snapshots semanales automáticos)	Recuperación ante fallas del servidor completo.
Backups de datos	DigitalOcean Spaces (S3-compatible)	Export diario de <code>mysqldump</code> , fuera del Droplet.
Monitoreo	DO Monitoring + Alertas (gratis)	Alertas de CPU/RAM/disco/uptime, crítico al ser punto único de falla.

Guía de Despliegue

Pasos de alto nivel para el aprovisionamiento inicial. Es una guía de referencia (runbook), no un script automatizado — antes de ejecutarla en un entorno real se recomienda convertirla en un plan de implementación detallado.

1. **Crear el Droplet:** Ubuntu 24.04 LTS, plan de 2–4 GB RAM, con Docker preinstalado (imagen "Docker on Ubuntu" del marketplace de DigitalOcean) o instalando Docker + Docker Compose manualmente.
2. **Configurar DNS:** crear un registro A apuntando `fe-api.tuempresa.com` (o el subdominio elegido) a la IP pública del Droplet.
3. **Copiar el proyecto al servidor:** transferir el repositorio de `API_Hacienda` con los 4 parches de infraestructura ya validados en la PoC local (Dockerfile con repos de Debian archivados, entrypoint sin CRLF, `settings.php` configurado, carpetas `logs/errors/files` creadas).
4. **Agregar Caddy al `docker-compose.yml`** como servicio adicional, con un `Caddyfile` que enruta el dominio hacia el contenedor `php-apache` y gestiona TLS automáticamente.
5. **Configurar el Cloud Firewall:** permitir entrante solo 443 (HTTPS) y 22 (SSH, restringido a IPs de administración conocidas); bloquear el puerto de MariaDB (3306) de forma pública.
6. **Resolver la autenticación multi-cliente** (ver Sección 5) antes de aceptar el primer cliente real — este paso es bloqueante, no opcional.
7. **Configurar backups automatizados:** activar Droplet Backups y programar un cron con `mysqldump` diario subiendo a DigitalOcean Spaces.
8. **Configurar monitoreo y alertas** de uptime, CPU, RAM y disco en el panel de DigitalOcean.
9. **Apuntar cada instalación de Odoo cliente:** actualizar `110n_cr_fe.api_url` a la URL pública, y provisionar sus credenciales/API key correspondientes.

Seguridad y Certificados .p12

Cada cliente eventualmente cargará su certificado criptográfico .p12 (emitido por Hacienda vía ATV) y su PIN para poder firmar comprobantes. Al centralizar la API, estos archivos sensibles de **todos los clientes** residirán en un único servidor bajo tu responsabilidad.

Medida	Detalle
Cifrado en reposo	Volumen de Docker donde viven los .p12 cifrado a nivel de sistema de archivos (no solo el cifrado de infraestructura que DigitalOcean aplica por defecto).
Permisos restringidos	Solo el proceso de la API puede leer esos archivos; acceso SSH de administración no debe implicar acceso trivial a los certificados en texto plano.
HTTPS obligatorio	No es opcional: certificados y cédulas de clientes viajan por la red pública.
Base de datos aislada	MariaDB nunca expuesta a internet; solo accesible desde el contenedor de la API dentro de la red interna de Docker.
Rotación de PIN/credenciales	Procedimiento documentado para que un cliente pueda rotar su certificado sin intervención manual compleja.

Nota de escala

A este tamaño no es necesario un gestor de secretos dedicado (ej. HashiCorp Vault). Si el número de clientes crece significativamente, esa es una inversión razonable a reconsiderar (ver Sección 11).

SECCIÓN 9

Precios Estimados

Los precios de la nube cambian con frecuencia.

Las cifras de esta sección son estimaciones de referencia (USD/mes) para dimensionar el proyecto. Deben confirmarse en digitalocean.com/pricing al momento de contratar.

Ítem	Especificación	Costo estimado
Droplet (inicio)	2 GB RAM / 1 vCPU / 50 GB SSD	~\$12/mes
Droplet (con margen)	4 GB RAM / 2 vCPU / 80 GB SSD	~\$24/mes
Backups automáticos del Droplet	+20% del costo del Droplet	~\$2.40–\$4.80/mes
DigitalOcean Spaces	250 GB + 1 TB transferencia (backups de BD)	~\$5/mes
Cloud Firewall	Incluido	Gratis
DNS	Incluido	Gratis
Monitoreo + alertas básicas	Incluido	Gratis
Dominio propio	Registrador externo (anual)	~\$10–15/año (~\$1/mes)
Total estimado (inicio)	Droplet 2GB + backups + Spaces + dominio	~\$20/mes
Total estimado (con margen)	Droplet 4GB + backups + Spaces + dominio	~\$32/mes

Este costo es **independiente del número de clientes** dentro de un rango razonable inicial (gracias al modelo de tabla-por-inquilino de la Sección 5) — no se paga más por cada cliente nuevo hasta que el volumen de tráfico o almacenamiento requiera escalar el Droplet o pasar a un modelo distribuido (Sección 11).

SECCIÓN 10

Riesgos y Aspectos a Considerar

Riesgo	Impacto	Mitigación
Punto único de falla	Alto	Si el servidor cae, todos los clientes pierden la capacidad de facturar simultáneamente. Requiere monitoreo de uptime activo y un plan de recuperación documentado.
Endpoints sin autenticación (heredado de la PoC)	Alto	Bloqueante antes de producción — ver Sección 5.
Responsabilidad sobre datos sensibles de terceros	Medio-Alto	Certificados y cédulas de todos los clientes centralizados implican responsabilidad legal/reputacional directa ante una brecha.
Obligaciones de la licencia AGPL v3	Medio	Consultar asesoría legal antes del lanzamiento comercial (ver Sección 3.4).
Escalamiento de almacenamiento (BD)	Bajo (a corto plazo)	El patrón tabla-por-inquilino crece linealmente en número de tablas; monitorear cuando el catálogo de clientes sea grande.
Dependencia de un solo mantenedor	Medio	Documentar el runbook de despliegue y recuperación para que no dependa de una sola persona.

Ruta de Escalamiento Futuro

La arquitectura propuesta está pensada para arrancar simple. Cuando el número de clientes o el volumen de comprobantes lo justifiquen, existe una ruta de crecimiento clara sin rediseñar desde cero:

Paso 1 — Escalamiento vertical (primer recurso)

Redimensionar el Droplet a más RAM/CPU. Es la opción más simple y la primera a considerar; DigitalOcean permite hacerlo con poco tiempo de inactividad.

Paso 2 — Separar la base de datos

Migrar de MariaDB en contenedor a **DigitalOcean Managed Database** (MySQL gestionado), lo que reduce la carga operativa (parches, backups, alta disponibilidad) a cambio de un costo mensual adicional (desde ~\$15/mes en el plan más pequeño).

Paso 3 — Escalamiento horizontal

Agregar un **DigitalOcean Load Balancer** (~\$12/mes) frente a múltiples Droplets ejecutando el contenedor de la API, todos apuntando a la base de datos gestionada centralizada. Necesario solo si el tráfico supera lo que un solo servidor puede manejar cómodamente.

Paso 4 — Gestión de secretos dedicada

Si el catálogo de certificados .p12 crece mucho, evaluar un gestor de secretos dedicado en vez de archivos cifrados en disco.

Conclusión y Próximos Pasos

El stack **DigitalOcean + Docker Compose + Caddy + MariaDB** ofrece el mejor balance entre simplicidad operativa, costo predecible y reutilización del trabajo ya validado en la prueba de concepto local, para hospedar la API de CRLibre como servicio centralizado que atienda a múltiples clientes de Odoo.

Antes de considerar esta arquitectura lista para producción, quedan dos bloqueantes concretos identificados en este documento:

- ☐ Resolver la autenticación por cliente (eliminar los endpoints `users_openAccess` del tráfico público) — Sección 5.
- ☐ Obtener orientación legal sobre las obligaciones de la licencia AGPL v3 al operar esto como servicio de red comercial — Sección 3.4.

Una vez resueltos estos dos puntos, el siguiente paso natural es convertir la Sección 7 (Guía de Despliegue) en un plan de implementación detallado, task por task, siguiendo el mismo proceso usado para construir la PoC.